

Code-based PKE based on Binary Goppa Code

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



September 11 & 12, 2026

Table of Content

1 Goppa-Code

2 Code-Based Cryptography

The next part explains a specific construction of a code-based public-key encryption scheme.
This part will be excluded from the exam.

Binary Goppa-Code

An Error-correcting code for Code-based Cryptography

Why Goppa Codes?

Question

We want a code with two properties:

- 1 Efficient decoding for the legitimate receiver.
- 2 A decoding problem that is hard for an attacker without secret information.

Why Goppa Codes?

Question

We want a code with two properties:

- 1 Efficient decoding for the legitimate receiver.
- 2 A decoding problem that is hard for an attacker without secret information.

A particularly important family for code-based cryptography is the **binary Goppa code**.

Goppa Code

Algebraic structure + Efficient decoding

Goppa codes form the mathematical foundation of the original **McEliece cryptosystem**.

Question

What special structure allows a receiver to decode efficiently?

Idea Behind a Goppa Code

Let

$$\mathbb{F}_{2^m}$$

be a finite field.

Choose:

- a set of distinct support elements

$$L = (\alpha_1, \alpha_2, \dots, \alpha_n), \quad \alpha_i \in \mathbb{F}_{2^m},$$

- a polynomial

$$g(x) \in \mathbb{F}_{2^m}[x]$$

of degree t .

The polynomial $g(x)$ is called the **Goppa polynomial**.

We require

$$g(\alpha_i) \neq 0 \quad \text{for all } i.$$

Question

Why should a polynomial and a set of field elements define a code?

Idea Behind a Goppa Code

Let

$$\mathbb{F}_{2^m}$$

be a finite field.

Choose:

- a set of distinct support elements

$$L = (\alpha_1, \alpha_2, \dots, \alpha_n), \quad \alpha_i \in \mathbb{F}_{2^m},$$

- a polynomial

$$g(x) \in \mathbb{F}_{2^m}[x]$$

of degree t .

The polynomial $g(x)$ is called the **Goppa polynomial**.

We require

$$g(\alpha_i) \neq 0 \quad \text{for all } i.$$

Question

Why should a polynomial and a set of field elements define a code?

The answer comes from imposing an algebraic condition on the vectors $x = (x_1, \dots, x_n) \in \mathbb{F}_2^n$.

Construction of a Binary Goppa Code

Given $g(x)$ and

$$L = (\alpha_1, \dots, \alpha_n),$$

define the binary Goppa code

$$\boxed{\Gamma(L, g)}$$

as the set of vectors

$$c = (c_1, \dots, c_n) \in \mathbb{F}_2^n$$

satisfying

$$\sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)}.$$

Equivalently,

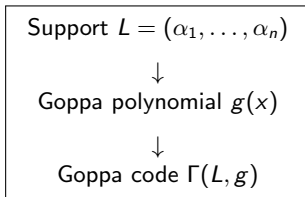
$$\boxed{\Gamma(L, g) = \left\{ c \in \mathbb{F}_2^n : \sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)} \right\}.$$

Important parameters

For a binary Goppa code with $\deg(g) = t$

A Small Structural View

The construction uses two ingredients:



where

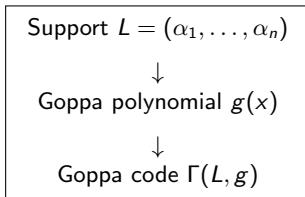
$$\alpha_i \in \mathbb{F}_{2^m}, \quad \deg(g) = t, \quad g(\alpha_i) \neq 0.$$

Question

What determines how many errors the code can correct?

A Small Structural View

The construction uses two ingredients:



where

$$\alpha_i \in \mathbb{F}_{2^m}, \quad \deg(g) = t, \quad g(\alpha_i) \neq 0.$$

Question

What determines how many errors the code can correct?

The degree t of the Goppa polynomial determines the designed error-correction capability:

Up to t errors can be corrected.

This is one of the important reasons Goppa codes are attractive for cryptography.

How Does Encoding Work?

A Goppa code is a **linear code**.

Therefore, once we obtain a generator matrix

$$G \in \mathbb{F}_2^{k \times n},$$

a message

$$m \in \mathbb{F}_2^k$$

is encoded as

$$\boxed{c = mG}.$$

Thus,

$$\mathbb{F}_2^k \xrightarrow{G} \Gamma(L, g) \subseteq \mathbb{F}_2^n.$$

Question

Does the Goppa polynomial itself directly perform the encoding?

How Does Encoding Work?

A Goppa code is a **linear code**.

Therefore, once we obtain a generator matrix

$$G \in \mathbb{F}_2^{k \times n},$$

a message

$$m \in \mathbb{F}_2^k$$

is encoded as

$$\boxed{c = mG}.$$

Thus,

$$\mathbb{F}_2^k \xrightarrow{G} \Gamma(L, g) \subseteq \mathbb{F}_2^n.$$

Question

Does the Goppa polynomial itself directly perform the encoding?

Not usually.

The Goppa polynomial and support define the code. We can then derive a generator matrix G for that code.

$$\boxed{\text{Code structure} \rightarrow G \rightarrow \text{Encoding}}$$

Generator Matrix of a Goppa Code

Let

$$H \in \mathbb{F}_2^{(n-k) \times n}$$

be a parity-check matrix for the Goppa code.

Then

$$\Gamma(L, g) = \{c \in \mathbb{F}_2^n : Hc^T = 0\}.$$

A generator matrix G satisfies

$$HG^T = 0.$$

The rows of G form a basis of the Goppa code.

Therefore every codeword has the form

$$c = mG.$$

Question

If we know G , what information does it reveal?

It completely specifies the linear code as a vector space.

This observation will become crucial when we construct a public-key encryption scheme.

Parity-Check Matrix of a Goppa Code

A parity-check matrix can be constructed from the support and Goppa polynomial.
A common representation over $\mathbb{F}_{2^m}$ is

$$H = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \frac{1}{g(\alpha_1)} & \frac{1}{g(\alpha_2)} & \cdots & \frac{1}{g(\alpha_n)} \\ \alpha_1 & \alpha_2 & \cdots & \alpha_n \\ \frac{\alpha_1}{g(\alpha_1)} & \frac{\alpha_2}{g(\alpha_2)} & \cdots & \frac{\alpha_n}{g(\alpha_n)} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_1^{t-1}}{g(\alpha_1)} & \frac{\alpha_2^{t-1}}{g(\alpha_2)} & \cdots & \frac{\alpha_n^{t-1}}{g(\alpha_n)} \end{bmatrix}.$$

Each entry lies in $\mathbb{F}_{2^m}$.

Expanding each field element into its m binary coordinates gives a binary parity-check matrix.

Question

What does $Hc^T = 0$ tell us?

c satisfies all parity constraints of the Goppa code.

Decoding a Goppa Code

Suppose a codeword c is transmitted through a noisy channel.
The receiver obtains

$$r = c + e,$$

where

$$\text{wt}(e) \leq t.$$

Question

What does the receiver need to recover?

The receiver needs to determine the error vector e .
Once e is known,

$$c = r - e.$$

For a Goppa code, the receiver computes the **syndrome**

$$s = Hr^T.$$

Since

Goppa Decoding: The Algebraic Idea

The receiver has

$$s = He^T.$$

The goal is to determine the error positions and values.

A Goppa decoder uses algebraic algorithms to recover the error vector.

Conceptually:

$$r \longrightarrow s \longrightarrow \text{error-locator polynomial} \longrightarrow \text{error positions} \longrightarrow e \longrightarrow c.$$

For binary Goppa codes, efficient algebraic decoding algorithms can correct up to t errors.

Question

Why can the legitimate receiver decode efficiently?

Goppa Decoding: The Algebraic Idea

The receiver has

$$s = He^T.$$

The goal is to determine the error positions and values.

A Goppa decoder uses algebraic algorithms to recover the error vector.

Conceptually:

$$r \longrightarrow s \longrightarrow \text{error-locator polynomial} \longrightarrow \text{error positions} \longrightarrow e \longrightarrow c.$$

For binary Goppa codes, efficient algebraic decoding algorithms can correct up to t errors.

Question

Why can the legitimate receiver decode efficiently?

Because the receiver knows the special algebraic structure defined by

$$\boxed{(L, g)}.$$

This structure enables the efficient Goppa decoding algorithm.

The Key Cryptographic Property

Now consider the two situations.

Legitimate Receiver

Knows

$$L, \quad g(x)$$

and therefore knows the special structure of the Goppa code.

Efficient decoding

Attacker

Sees a public representation that should hide the special Goppa structure.

Decoding is hard

Answer

The property providing easy decoding with secret information is the **hidden algebraic structure of the Goppa code**, specifically the knowledge of the support L and Goppa polynomial $g(x)$ that enables the efficient Goppa decoding algorithm.

Why Is This Useful for Public-Key Cryptography?

We now have an interesting situation:

Secret knowledge of (L, g)



Efficient decoding

while we want the public representation to hide this structure:

Public representation



Hard decoding problem

Question

Can we transform the generator matrix of the Goppa code so that:

- Bob can still use his secret Goppa structure,
- Eve sees only a disguised generator matrix?

Goppa Code $\rightarrow$ Hidden Structure $\rightarrow$ McEliece PKE

From Goppa Code to McEliece

Let G be a generator matrix of the secret Goppa code.

We disguise it using invertible transformations:

$$G_{\text{pub}} = SGP$$

where S is an invertible matrix and P is a permutation matrix.

The public key contains

$$G_{\text{pub}}.$$

The secret key contains the information needed to recover the original Goppa structure.

Question

What is the central cryptographic assumption now?

Given the public representation, decoding a noisy word is hard.

Goppa decoding $\implies$ Trapdoor decoding $\implies$ Code-Based PKE

Goppa-Code: Example

A Toy Binary Goppa Code

Question

Can we construct a very small Goppa code and work through the entire process by hand?

We work over the finite field

$$\mathbb{F}_4 = \{0, 1, \alpha, \alpha + 1\}, \quad \alpha^2 = \alpha + 1.$$

Choose the support

$$L = (0, \alpha, \alpha + 1)$$

and choose the Goppa polynomial

$$\boxed{g(x) = x + 1}.$$

Since

$$g(0) = 1, \quad g(\alpha) = \alpha + 1, \quad g(\alpha + 1) = \alpha,$$

we have

$$g(\alpha_i) \neq 0 \quad \forall \alpha_i \in L.$$

Thus we obtain the binary Goppa code

$$\boxed{\Gamma(L, g)}.$$

Here

$$n = 3, \quad \deg(g) = t = 1.$$

Toy Goppa Code: Constructing the Parity-Check Matrix

For a Goppa code, a parity-check matrix over $\mathbb{F}_4$ can be written as

$$H = \begin{bmatrix} \frac{1}{g(\alpha_1)} & \frac{1}{g(\alpha_2)} & \frac{1}{g(\alpha_3)} \end{bmatrix}.$$

For our choice,

$$g(0) = 1, \quad g(\alpha) = \alpha + 1, \quad g(\alpha + 1) = \alpha.$$

Using

$$\frac{1}{\alpha + 1} = \alpha, \quad \frac{1}{\alpha} = \alpha + 1,$$

we obtain

$$H = \begin{bmatrix} 1 & \alpha & \alpha + 1 \end{bmatrix}.$$

Representing each element of $\mathbb{F}_4$ using two binary coordinates gives

$$H_{\mathbb{F}_2} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Finding the Codewords

A binary vector

$$c = (c_1, c_2, c_3)$$

is a codeword if

$$H_{\mathbb{F}_2} c^T = 0.$$

Therefore,

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

This gives

$$c_1 + c_3 = 0, \quad c_2 + c_3 = 0.$$

Hence

$$c_1 = c_3, \quad c_2 = c_3.$$

Therefore the code contains only

Encoding in the Toy Goppa Code

The generator matrix is

$$G = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}.$$

For a message

$$m \in \mathbb{F}_2,$$

encoding is simply

$$\boxed{c = mG}.$$

Therefore,

$$m = 0 \implies c = 000,$$

and

$$m = 1 \implies c = 111.$$

Hence

Transmission Through a Noisy Channel

Suppose Alice wants to send

$$m = 1.$$

The encoded codeword is

$$c = 111.$$

Suppose one bit is flipped during transmission:

$$111 \longrightarrow 101.$$

Bob receives

$$r = 101.$$

Question

How can Bob determine that an error occurred?

Bob computes the syndrome

Toy Decoding: Locating the Error

For a single-bit error,

$$r = c + e_i,$$

where e_i has a 1 in position i .

The syndrome is

$$s = Hr^T = He_i^T.$$

Therefore, the syndrome is exactly the corresponding column of H .

For our code,

$$H = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Hence

Error position	Syndrome
1	$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$

What Did We Actually Use for Decoding?

Our decoding process was

$$r \longrightarrow s = Hr^T \longrightarrow \text{error position} \longrightarrow e \longrightarrow c = r - e \longrightarrow m.$$

For the toy code:

$$101 \xrightarrow{H} 01 \longrightarrow \text{position 2} \longrightarrow 010 \longrightarrow 111 \longrightarrow 1.$$

Question

Why was it easy to decode?

In this toy example, the parity-check matrix directly reveals the error position.

For a general Goppa code, the corresponding algebraic structure enables efficient decoding of up to t errors.

Goppa structure $\Rightarrow$ Efficient algebraic decoding

Where Is the “Secret” in Goppa Decoding?

Question

If the Goppa code has an efficient decoder, why can we not simply publish everything?

The efficient decoder relies on the special algebraic description

$$(L, g)$$

where

$$L = (\alpha_1, \dots, \alpha_n)$$

is the support and

$$g(x)$$

is the Goppa polynomial.

For the legitimate receiver:

$$(L, g) \Rightarrow \text{special algebraic structure} \Rightarrow \text{efficient decoding}$$

From the Toy Example to McEliece

Our toy example has

$$G = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}.$$

In a McEliece-type construction, Bob starts with a generator matrix G of a much larger binary Goppa code.

He then disguises it using secret transformations.

Conceptually,

$$G \xrightarrow{\text{secret transformations}} G_{\text{pub}}.$$

Bob keeps the Goppa structure and the transformations secret.

Alice sees only

$$G_{\text{pub}}$$

and introduces a random error:

$$c = mG_{\text{pub}} + e.$$

Code-Based Cryptography

Why Code-Based Cryptography?

Question

Error-correcting codes are designed to correct errors. How could they possibly help us build public-key cryptography?

Why Code-Based Cryptography?

Question

Error-correcting codes are designed to correct errors. How could they possibly help us build public-key cryptography?

Consider

$$y = c + e,$$

where c is a valid codeword and e is a small error vector. If we know the appropriate decoding structure, recovering c can be efficient.

Cryptographic idea

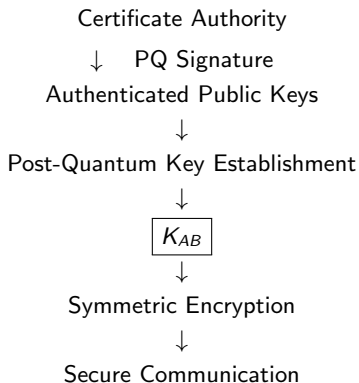
Can we construct a system where:

- decoding is easy for the legitimate receiver,
- decoding is computationally hard for an attacker?

Efficient decoding for Bob

Hard decoding for Eve

Cryptographically Secure Channel



Broader system view

Once we have a secure PKI and a secure post-quantum key-establishment mechanism, a client and server can establish a secure channel and then communicate securely, including while executing privacy-sensitive AI/ML algorithms.

How can we use an error-correcting code to establish a shared secret between two parties?

Our path

Error-Correcting Codes → Decoding Problem → Code-Based PKE → KEM → Shared Secret

Let's build it from the decoding problem.

Textbook:

- 1 Daniel J. Bernstein, Johannes Buchmann, and Erik Dahmen. *Post-Quantum Cryptography*. Springer, 2009. ISBN: 9783540887010.

Acknowledgment:

- Some figures in this work were generated with the assistance of ChatGPT (OpenAI). It is used to decorate contents as well.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.